

Chapter 1. Software Quality Assurance Fundamentals

1.1. Definition of Quality, Quality Assurance, Quality Control, Difference between QA and QC, Software Quality Assurance Challenges

1.2. Software Quality Assurance, SQA Planning & Standards (ISO 9000)

1.3. SQA Activities

1.4. Building Blocks of SQA

1.5. Software Quality factors

1.6. Software Reliability & Reliability Measurement Factors: ROCOF, MTTF, MTTR, MTBF, POFOD, Availability

1.1. Definition of Quality, Quality Assurance, Quality Control, Difference between QA and QC, Software Quality Assurance Challenges

What is Quality ?

Quality is how well a product or service meets the needs, expectation or standards of its users, ensuring it is reliable, functional & free from defects

What is Quality Assurance?

- Assurance is a positive declaration on a product or service.
- It is all about the product which would work well.
- It provides a guarantee which would work without any problem according to expectation & requirements.
- Quality assurance is a responsibility of each member of the development team.
- Quality assurance is also called QA testing.
- Quality assurance is a method of making the software application with fewer defects &

mistakes when it is finally released to the end user.

- It recognizes defects in the development process.
- Quality assurance is completed before quality control

What is Quality Control ?

Quality control involves the activities & techniques used to inspect & verify that the product meets the specified quality standards

Quality control is product-oriented, focusing on identifying & fixing defects in final products

It focuses on examination of the quality of the end product & the final outcome rather than focusing on the processes used to create the product

Difference between QA and QC

What is Quality Control ?

Quality control involves the activities & techniques used to inspect & verify that the product meets the specified quality standards

Quality control is product-oriented, focusing on identifying & fixing defects in final products

It focuses on examination of the quality of the end product & the final outcome rather than focusing on the processes used to create the product

Difference between QA and QC

Quality Assurance /QA	Quality Control / QC
QA focuses on providing assurance that the quality requested will be achieved	QC focuses on fulfilling the quality requested
QA is the technique of managing quality	QC is the technique to verify quality
QA focuses on preventing defect by improving optimizing the processes used to develop software	QC focuses on identifying & fixing defect in final product
QA is process oriented	QC is product oriented
QA prevent & minimizes software defects during development	QC identifies & fixes the defects that exist in the final product

development time	development
QA is a managerial tool	QC is corrective tool
QA is performed before QC	QC is performed after the QA activity is done
QA is preventive technique	QC is corrective technique
QA is low level activity that identify an error & mistake	QC is high level activity that identify an error that QA cannot
QA is less time consuming	QC is more time consuming process
Eg. Verification	Eg. Validation

Software Quality Assurance (SQA) Challenges:

- **Changing Requirements:**
Frequent changes in requirements can disrupt planned processes and make it harder to ensure quality.
- **Tight Deadlines:**
Limited time for development and testing can lead to incomplete testing or skipping quality checks.
- **Resource Constraints:**
Insufficient resources, such as skilled personnel, tools, or budget, can affect the quality assurance process.
- **Integration Issues:**
Ensuring seamless integration of different modules and components can be complex, especially in large systems.
- **Communication Gaps:**
Miscommunication between teams, such as developers, testers, and clients, can lead to misunderstandings about requirements or priorities.
- **Complex Systems:**
Testing complex systems with numerous dependencies and interactions can be challenging and time-consuming.
- **Lack of Proper Tools:**
Without the right testing tools or automation, the QA process can become inefficient and

error-prone.

➤ **Emerging Technologies:**

Rapid advancements in technology require QA teams to stay updated, which can be difficult.

➤ **Ensuring Security:**

Identifying and addressing vulnerabilities to ensure data security and compliance is a major challenge in modern software.

➤ **Cross-Platform Compatibility:**

Ensuring the software works across various devices, operating systems, and browsers adds complexity to the QA process.

1.2. Software Quality Assurance, SQA Planning & Standards (ISO 9000)

Definition:

SQA is a systematic process of evaluating the quality of software during its development lifecycle. It ensures that the software meets the specified requirements and standards, reducing defects and improving the overall reliability of the product.

Objective:

- Make sure the work is done as per the rules and guidelines.
- Find and fix issues as soon as possible during development.
- Ensure the software works well, lasts long, and performs reliably.
- Deliver a product that meets customer expectations and earns their confidence.

SQA Planning

SQA Planning is the process of organizing and defining the activities, resources, and methods to ensure software quality during its development. It serves as a roadmap to prevent defects, reduce risks, and deliver reliable software.

Key Steps in SQA Planning

➤ **Set Quality Goals:**

Define what "quality" means for the project.

Example: "The software must have less than 1 critical defect per 1,000 lines of code."

➤ **Identify Standards and Guidelines:**

Choose industry and organizational standards, like ISO 9001 or CMMI.

Establish coding, testing, and documentation standards.

➤ **Plan SQA Activities:**

Include tasks such as:

- Reviews (design, code, and test plans).
- Audits to ensure processes are followed.
- Testing strategies (unit, integration, system testing).

➤ **Allocate Resources:**

Assign people, tools, and time needed for quality assurance activities.

Example: Use tools like JIRA for defect tracking or Selenium for automated testing.

➤ **Define Roles and Responsibilities:**

SQA Team: Monitor and enforce quality practices.

Developers: Write quality code and fix defects.

Testers: Verify software functionality and reliability

➤ **Develop Metrics and Reporting Plans:**

Track key performance indicators (KPIs) like:

- Defect density (defects per 1,000 lines of code).
- Test coverage percentage.
- Average time to fix defects.

Use these metrics to monitor progress and report quality status.

➤ **Risk Management:**

Identify potential risks to quality, such as:

- Changing requirements.
- Insufficient testing resources.

Develop strategies to mitigate these risks.

➤ **Create a Schedule:**

Align quality activities with project milestones.

Example: Conduct code reviews after each development phase.

➤ **Establish Communication Channels:**

Ensure clear reporting of defects, progress, and quality metrics to stakeholders.

➤ **Document the Plan:**

Prepare a formal SQA plan that includes all of the above details.

Share it with the team and stakeholders for alignment and approval

Benefits of SQA Planning

- **Early Problem Detection:** Identify and resolve issues before they grow.
- **Improved Product Quality:** Deliver reliable and efficient software.
- **Risk Reduction:** Minimize potential risks to the project's success.
- **Cost Efficiency:** Reduce rework and defect correction costs.
- **Stakeholder Satisfaction:** Ensure customer requirements are met.

ISO 9000

ISO 9000 is a family of international standards focused on quality management systems (QMS). It provides guidelines and requirements for organizations to ensure consistent quality in products and services.

ISO 9000 focuses on how work is done (process quality), not on the product itself, to ensure consistent quality and customer satisfaction.

ISO = International Organization for Standardization

9000 = Series number for quality management standards

So, ISO 9000 ensures that an organization follows standardized, documented, and continuously improving processes to deliver quality products or services.

Key Principles of ISO 9000:

1. **Customer Focus:** Understanding and meeting customer needs.
2. **Leadership:** Establishing unified goals for quality improvement.
3. **Process Approach:** Managing activities as interconnected processes for efficiency.
4. **Continuous Improvement:** Striving for ongoing quality enhancement

ISO 9001:

The most widely used standard under ISO 9000, specifying requirements for establishing and maintaining a QMS. It emphasizes process documentation, defect prevention, and customer satisfaction.

Benefits of ISO 9000 in Software Development:

- Standardized processes and documentation.
- Improved product quality and reliability.
- Enhanced customer trust and market competitiveness.

Six Sigma

Six Sigma is a method used by companies to improve the quality of their products or services by reducing mistakes and making processes more efficient. It focuses on finding and fixing problems to ensure things work as perfectly as possible.

“Sigma” is a term from statistics that measures how much variation (or mistakes) there are in a process.

Key Ideas of Six Sigma

1. **Focus on Customer Needs:** Understand what the customer wants and make sure the product or service meets those expectations.
2. **Reduce Errors:**
 - Identify where mistakes happen in the process and work to eliminate them.
 - Example: If a food delivery app often delivers late, Six Sigma helps find and fix the reasons for delays.
3. **Use Data and Numbers:**
 - Decisions are based on facts and measurements, not guesses.
 - Example: Track how many orders are late and what causes the delay.
4. **Continuous Improvement:** Always look for ways to make things better, even if the process is already working well.

Six Sigma Process (DMAIC)

Six Sigma uses a simple 5-step method called **DMAIC** to solve problems:

1. **Define:**
What's the problem? What does the customer want?
Example: Customers complain about late deliveries.
2. **Measure:**
Collect data to understand how often the problem happens.
Example: Measure how many deliveries are late and by how much time.
3. **Analyze:**
Find out why the problem is happening.
Example: Are deliveries late because of traffic, wrong addresses, or slow preparation?
4. **Improve:**
Fix the issues causing the problem.
Example: Add more delivery drivers or use better route-planning software.
5. **Control:**
Make sure the improvements continue to work in the long term.
Example: Regularly check delivery times to ensure they stay on track.

1.3. SQA Activities

- **SQA Management Plan:**
Create a plan to ensure quality throughout the project. Decide the best software engineering activities for the project and check if the SQA team has the required skills.
- **Set Checkpoints:**
Define specific points in the project to evaluate progress. Use collected data to measure the project's performance at each checkpoint.
- **Measure Change Impact:**
When fixing an error, it might cause new errors. Keep track of how changes affect the project and test the fixes to ensure compatibility with the whole system.
- **Use Multiple Testing Strategies:**
Don't rely on just one testing method. Use different testing approaches to find as many issues as possible.
- **Manage Good Relations:**
Build good relationships with other teams, like developers. Poor relationships can negatively affect the project. Avoid conflicts and focus on collaboration.
- **Maintain Records and Reports:**

Document everything, including test cases, defects, changes, and test cycles. Share these records with stakeholders for better understanding and future reference.

➤ **Review Software Engineering Activities:**

Check and document the processes used in the project. Make sure the software is being developed correctly and meets quality standards.

➤ **Formalize Deviation Handling:**

Record and track anything that doesn't go as planned. Follow clear steps to address these issues in an organized way.

1.4. Building Blocks of SQA

➤ **Software Engineering Standards:** These are predefined rules and guidelines (e.g., coding standards, testing standards) that ensure quality and consistency in the development process.

➤ **SQA Processes:** These include activities like reviews, audits, and inspections to ensure the software meets quality standards at every stage of development.

➤ **Verification and Validation:**

Verification: Checks if the product is being built the right way (e.g., are we following processes correctly?).

Validation: Checks if the right product is being built (e.g., does it meet user needs?).

➤ **Testing Strategies:** Use various testing methods (unit testing, integration testing, system testing, etc.) to identify defects and ensure the software works as expected.

➤ **Error Reporting and Tracking:** A system to report, track, and manage errors or defects found during development and testing.

➤ **Risk Management:** Identify potential risks that could affect software quality and develop strategies to minimize or avoid them.

➤ **Configuration Management:** Manage changes to software components systematically to ensure consistency and prevent errors due to untracked changes.

➤ **SQA Tools and Automation:** Use tools for activities like testing, version control, and performance monitoring to enhance efficiency and accuracy.

➤ **Training and Team Skills:** Ensure the SQA team has the skills and knowledge needed to carry out their roles effectively through proper training.

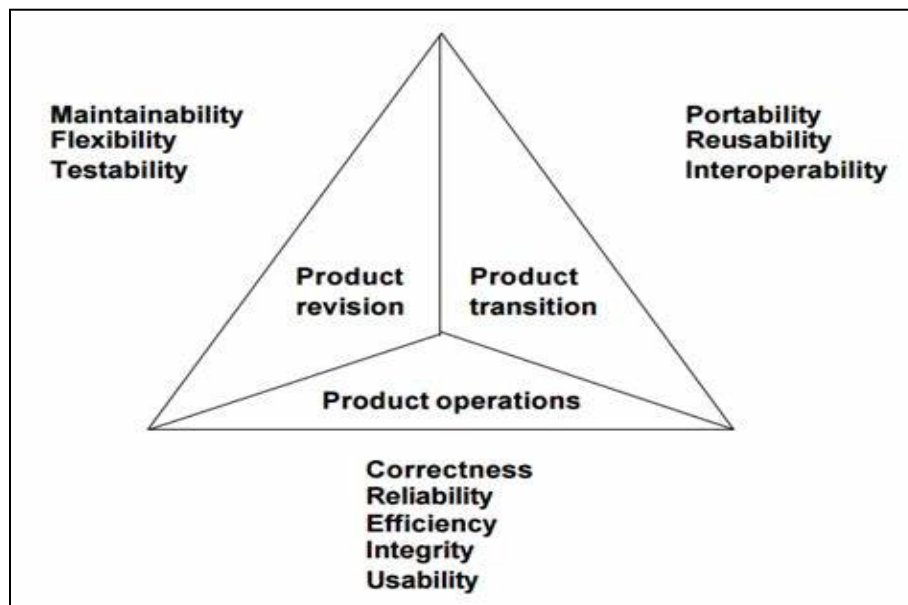
➤ **Metrics and Reporting:** Collect data on various aspects of the project (e.g., defect density,

test coverage) and use it to monitor progress and improve quality.

- **Management Commitment:** Strong support and commitment from management to ensure that quality is prioritized throughout the software lifecycle.

1.5. Software Quality factors

Software Quality Factors refer to the attributes or characteristics of software that determine its ability to meet user requirements and expectations



McCall's Triangle of Quality

1. Product Operation

This section focuses on how well the software performs during use. The factors are:

- **Correctness:** Ensures the software delivers the expected functionality and accurate results, meeting user requirements.
Example: A fee system correctly shows the paid amount and remaining balance.
- **Reliability:** Ensures the software can operate without failure for a specified period under normal conditions.
Example: An online exam system runs without crashing during the exam.
- **Efficiency:** Measures how well the software uses system resources like memory, CPU, and time to deliver optimal performance.

Example: A mobile app opens quickly without slowing the phone.

- **Integrity:** Refers to the software's ability to protect data from unauthorized access or corruption, ensuring security.

Example: Only teachers can edit student marks using secure login.

- **Usability:** Determines how easy it is for users to interact with the software, focusing on user-friendliness and intuitive design.

Example: A college website with simple menus and clear navigation.

2. Product Revision

This section highlights the software's ability to be improved or updated after its initial release. The factors are:

- **Maintainability:** Ensures the software is easy to fix, update, or modify when bugs are found or changes are needed.

Example: A bug in attendance software is corrected quickly.

- **Flexibility:** Refers to the software's ability to adapt to new requirements or environments without major rework.

Example: Adding online assignments to an existing LMS.

- **Testability:** Ensures the software can be easily tested to identify defects and validate its functionality during updates.

Example: Separate testing of login and payment modules.

3. Product Transition

This section focuses on the software's ability to adapt to new platforms, systems, or projects. The factors are:

- **Portability:** The software should run on different hardware or operating systems with minimal changes.

Example: A Java program runs on Windows and Linux.

- **Reusability:** Refers to the ability to reuse parts of the software (e.g., modules or code) in other projects, saving time and resources.

Example: Using the same login module in multiple systems.

- **Interoperability:** Ensures the software can communicate and work seamlessly with other systems or applications.

Example: College ERP connecting with an online payment gateway.